



CMPE491 – Analysis Report

HorusEye – AI-Based Exam Proctoring System

2025

Team Members:

Çağla Abazaoğlu – 10051217210

Gizem Nur İpek – 46162949900

Taha Kürşat Öztürk – 15709032030

Ali Sahil – 14596166750

Tuğba Hilal Kırer – 10015328662

Contents

1. Introduction	1
2. Current System	2
3. Proposed System	3
3.1 Overview	3
3.2 Functional Requirements	3
3.3 Nonfunctional Requirements	4
3.4 Pseudo Requirements	4
3.5 System Models	5
3.5.1 Scenarios	5
3.5.2 Use Case Model	5
3.5.3 Object and Class Model	6
3.5.4 Dynamic Models	6
3.5.5 User Interface – Navigational Paths and Screen Mock-Ups	7
3.5.6 System Architecture Diagram	8
4. Glossary	9
5. References	10

1. Introduction

HorusEye is an AI-driven exam proctoring system designed to ensure academic integrity in both online and physical examinations. As hybrid and remote education becomes increasingly common, traditional human-based invigilation struggles with scalability, accuracy, and objectivity. HorusEye addresses these challenges by providing real-time, automated, and intelligent monitoring that ensures fairness and reliability throughout the exam process.

Using computer vision, deep learning, and behavioral analysis, the system monitors exam environments through strategically placed cameras and detects suspicious actions such as abnormal eye movement, device usage, or unauthorized communication. Detected incidents are instantly reported to supervisors via a secure dashboard, along with time-stamped visual evidence for review. Built on a scalable microservice architecture using Python, OpenCV, TensorFlow, and YOLOv8, HorusEye offers high-performance real-time processing and cross-platform accessibility. With GDPR/KVKK-compliant data handling, privacy protection, and bias reduction mechanisms, the system prioritizes ethical AI principles and responsible surveillance.

Ultimately, HorusEye presents a future-ready digital invigilation solution that blends technology, fairness, and transparency to create a secure and trustworthy exam experience.

2. Current System

Today, exam security is mostly maintained through physical invigilation and basic online monitoring tools. However, these methods fail to keep up with modern technological needs and are insufficient for hybrid or large-scale examination environments. Especially in remote or crowded exam settings, the current system shows several major limitations:

Limited Human Monitoring: Invigilators struggle to observe multiple students simultaneously and often fail to detect subtle behaviors such as eye diversion, whispering, minor hand gestures, or mobile device usage.

Lack of Standardization: Since each invigilator may interpret behaviors differently, the level of supervision varies, leading to inconsistency and potential unfairness.

Inadequate Online Monitoring: Conventional webcam-based tools mainly record video but cannot analyze behavior, detect objects, track gaze, or provide real-time alerts.

Lack of Evidence Collection: Most current systems do not provide timestamped video evidence, making it difficult to verify and evaluate suspicious incidents after the exam.

Scalability Issues: Assigning enough invigilators for large-scale or multi-location exams is both costly and logistically challenging, making traditional supervision methods unsustainable.

Privacy and Legal Compliance Challenges: Some existing tools do not fully comply with GDPR/KVKK data protection regulations, which raises ethical and legal concerns.

Due to these limitations, the current examination monitoring system lacks reliability, efficiency, and sustainability, failing to meet the modern requirements of digital education. At this point, HorusEye aims to eliminate these weaknesses by providing an intelligent, automated, real-time, and ethical exam proctoring solution.

3. Proposed System

3.1 Overview

HorusEye is an AI-powered proctoring system that uses computer vision and deep learning to monitor exams in real time, detect suspicious behaviors, and provide automated alerts. By combining video processing, gaze tracking, object recognition, and behavior analysis, it offers a scalable and objective alternative to traditional human supervision.

The system analyzes live camera feeds to identify actions such as abnormal eye movement, whispering, device usage, or unauthorized communication. Detected incidents are logged with timestamped video clips and sent to supervisors via a secure web-based dashboard.

Built on a Python-based microservice architecture using OpenCV, YOLOv8, TensorFlow, Docker, and PostgreSQL, HorusEye delivers real-time processing, modular scalability, and an intuitive interface for managing alerts, monitoring multiple exam rooms, and reviewing post-exam reports.

3.2 Functional Requirements

Real-Time Behavior Detection:

The system should analyze live camera streams to automatically identify suspicious activities such as eye diversion, mobile device usage, whispering, or seeking external assistance.

Instant Alert and Notification System:

When suspicious behavior is detected, the system should immediately notify supervisors via visual markers, dashboard alerts, or audible warnings through the web interface.

Event Logging with Evidence:

The system should record each detected incident with time-stamped video clips, enabling supervisors to review and verify events after the examination.

AI-Powered Activity Recognition:

The system should utilize deep learning and computer vision techniques to detect objects such as phones, paper, or earbuds, as well as analyze head and eye movements.

Post-Exam Behavior Analysis and Reporting:

The system should generate detailed reports summarizing suspicious behaviors, timestamps, frequency of incidents, risk levels, and behavior categories after the exam.

Support for Real-Time and Offline Modes:

The system should operate in both real-time monitoring mode during live exams and offline mode by analyzing recorded exam videos afterward.

3.3 Nonfunctional Requirements

Accuracy:

The system shall achieve a minimum detection accuracy of 85% under standard lighting conditions.

Platform Compatibility:

The system shall run on Windows, macOS, and Linux environments, supporting both desktop and web access.

Security:

All data transmissions shall be secured using HTTPS, and sensitive information shall be encrypted in storage and during transfer.

Scalability:

The system shall support multiple exam rooms, large groups of students, and parallel monitoring without performance degradation.

Privacy and Legal Compliance:

The system shall comply with GDPR/KVKK data protection guidelines to ensure ethical and lawful processing of visual data.

Usability:

The dashboard interface shall be intuitive, responsive, and easy to navigate for supervisors, regardless of technical expertise.

3.4 Pseudo Requirements

Ethical Monitoring:

The system will focus on fairness, transparency, and unbiased decision-making by minimizing false alarms and respecting student privacy.

Environmental Adaptability:

The system will be designed to operate efficiently under varying environmental conditions such as lighting differences, camera placements, and seating layouts.

User Inclusivity:

The dashboard will be accessible and usable by academic staff with different levels of technical experience, from basic users to advanced administrators.

3.5 System Models

3.5.1 Scenarios

Scenario 1:

During a physical exam, the system monitors students via classroom cameras. A student repeatedly looks sideways and whispers. The AI model identifies this as suspicious behavior, logs the incident with a timestamped video clip, and sends an alert to the proctor through the web dashboard.

Scenario 2:

An instructor accesses the web dashboard after the exam to review all flagged incidents. The system displays video evidence, risk levels, and behavioral patterns for each student, allowing the instructor to evaluate the case objectively.

Scenario 3:

In an online examination, the system detects a student using a mobile phone through object recognition. It instantly notifies the supervisor, and the event is recorded for post-exam analysis.

Scenario 4:

A student's gaze frequently shifts away from the screen during a remote exam. The system classifies this as abnormal eye movement and logs multiple low-risk alerts, which are summarized in the post-exam behavior report.

3.5.2 Use Case Model

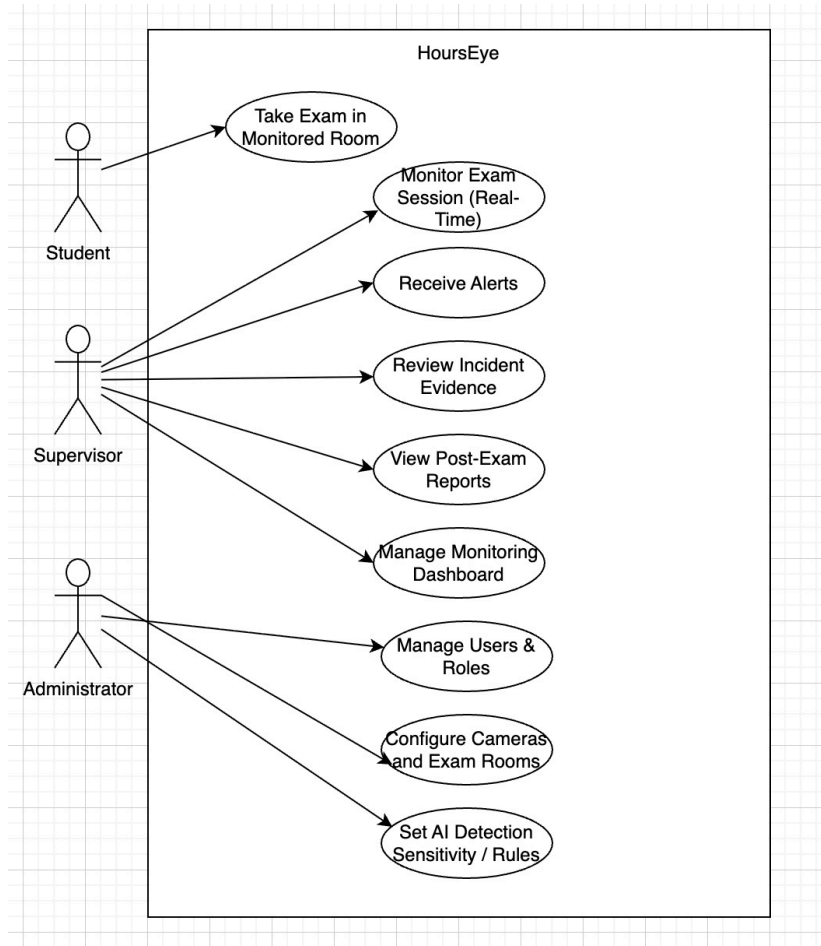
Actors:

- **Student** (Exam Participant)
- **Supervisor / Proctor**
- **HorusEye System**

Primary Use Cases:

- **UC-01: Monitor Exam Session (Real-Time)**
The system observes student behavior through live video streams and analyzes it using AI.
- **UC-02: Detect Suspicious Behavior**
The system detects behaviors such as phone usage, gaze diversion, whispering, unauthorized communication, or object appearance.
- **UC-03: Generate Alerts**
When suspicious activity occurs, the system sends alerts to supervisors via the dashboard or notification system.
- **UC-04: Record Incidents with Evidence**
The system captures and stores timestamped video clips of detected events for later review.

- **UC-05: Review Post-Exam Reports**
Proctors can review detailed post-exam analysis reports, including behavior frequency, severity, and evidence clips.
- **UC-06: Manage Monitoring Dashboard**
Supervisors can monitor multiple exam rooms, view live feeds, filter alerts, and access recorded data.



3.5.3 Object and Class Model

Classes:

User

Represents individuals who interact with the system, such as supervisors and administrators. Includes role, credentials, access permissions, and activity logs.

Student

Represents an exam participant being monitored during the exam. Includes identity details, seat number, and participant metadata.

ExamSession

Represents a specific exam session. Includes session ID, date, duration, associated students, assigned rooms, and camera mappings.

ExamRoom

Represents a physical exam room. Includes room name, capacity, lighting conditions, camera list, and assigned exam sessions.

Camera

Represents a surveillance device installed in an exam room. Includes IP address, location, resolution, and video stream configuration.

VideoStream

Represents live video feed captured from a camera and forwarded to AI modules. Includes stream ID, format, frame rate, and processing status.

DetectionEngine (AIProcessor)

Represents the AI engine responsible for analyzing video streams using YOLOv8, OpenCV, and TensorFlow. Generates incident objects based on detected events.

Incident

Represents detected suspicious behavior such as abnormal gaze, device usage, whispering, or unauthorized communication. Includes timestamp, student ID, event type, risk level, and video clip reference.

Alert

Represents system-generated notifications sent to supervisors when suspicious activity is detected. Includes alert type (visual, dashboard, sound), status, and related incident ID.

RiskAssessment

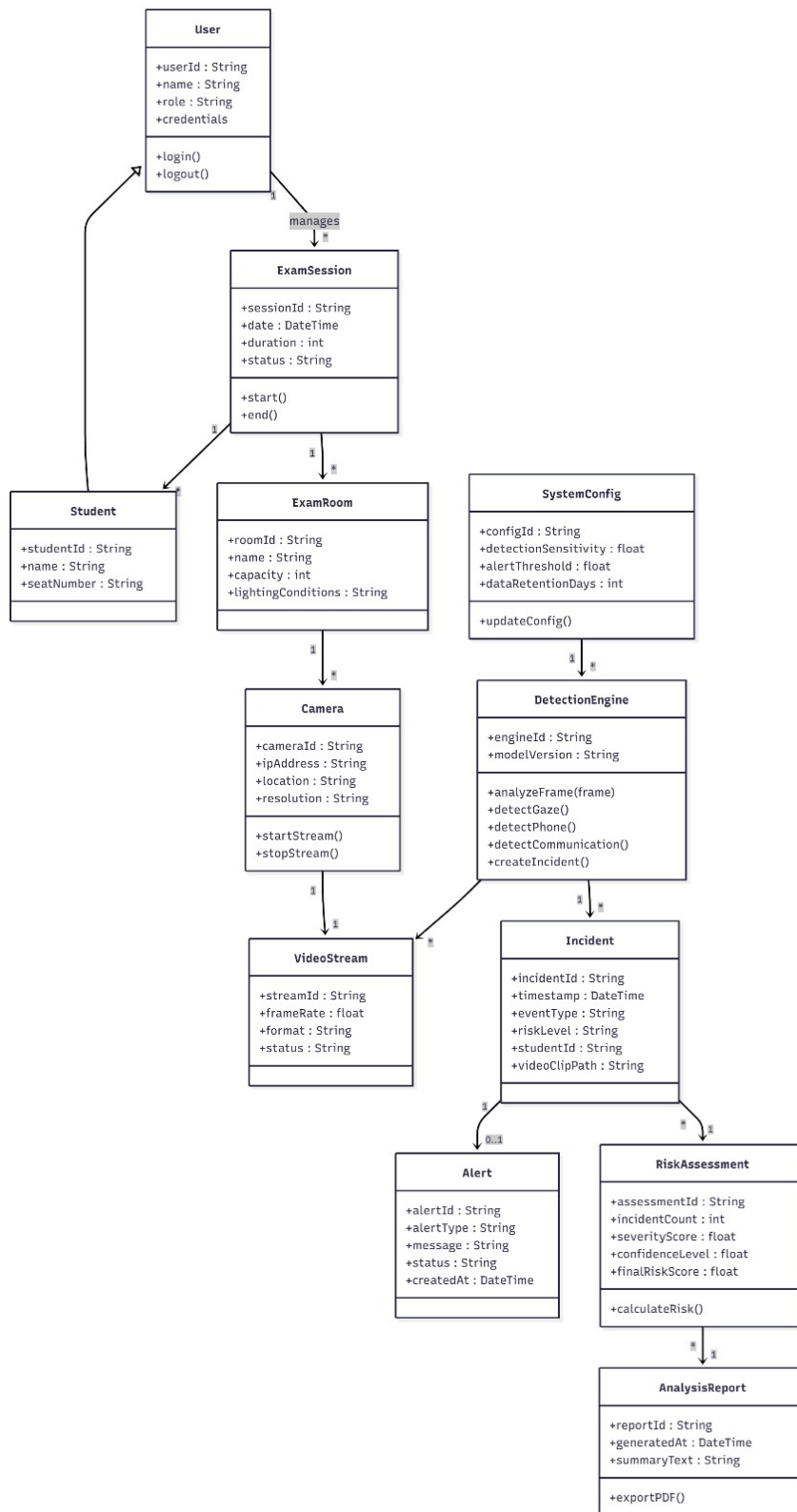
Represents the calculation of risk levels for students or exam sessions. Includes incident count, severity metrics, AI confidence levels, and final risk scoring.

AnalysisReport

Represents a post-exam behavioral analysis report. Includes summary of incidents, student-specific risk levels, behavioral patterns, and video evidence.

SystemConfig (Settings)

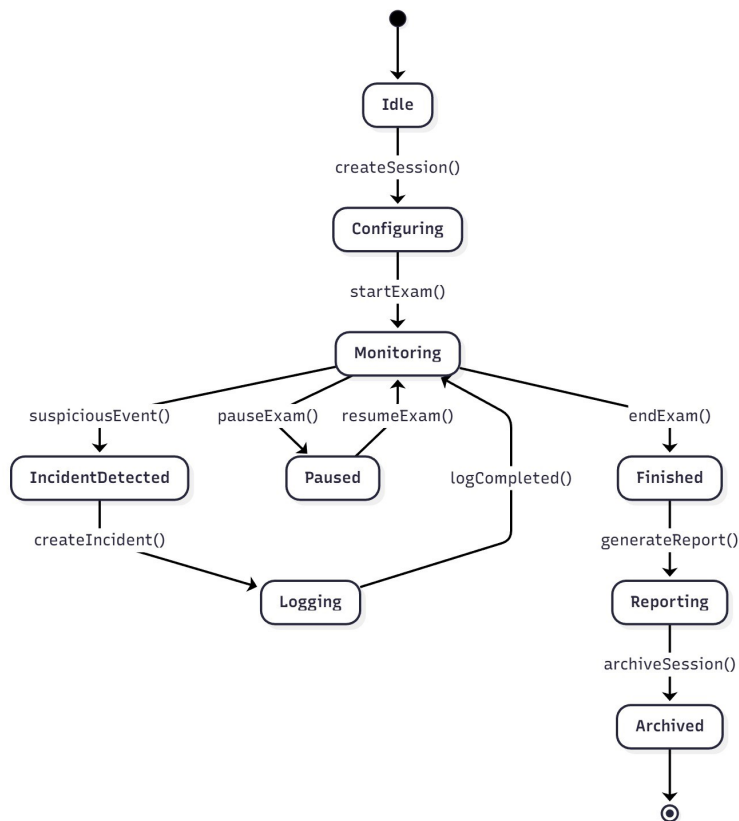
Represents global configuration settings such as detection sensitivity, alert thresholds, data retention policies, and privacy compliance parameters.



3.5.4 Dynamic Models

Activity Diagram:

Illustrates the process flow of a monitored exam session. The diagram shows how the system starts the exam monitoring, captures video streams from cameras, analyzes student behavior using AI models, detects suspicious activities, generates real-time alerts for supervisors, logs incidents with timestamped video evidence, and finally produces post-exam analysis reports for review.



3.5.5 User Interface - Navigational Paths and Screen Mock-Ups

Screens:

Login Screen: Allows supervisors and administrators to securely access the system.

Dashboard (Live Monitoring): Displays live camera feeds, list of active exam rooms, monitored students, and real-time system status.

Alerts Panel: Shows instant notifications of detected suspicious behaviors with incident type, timestamp, and risk level.

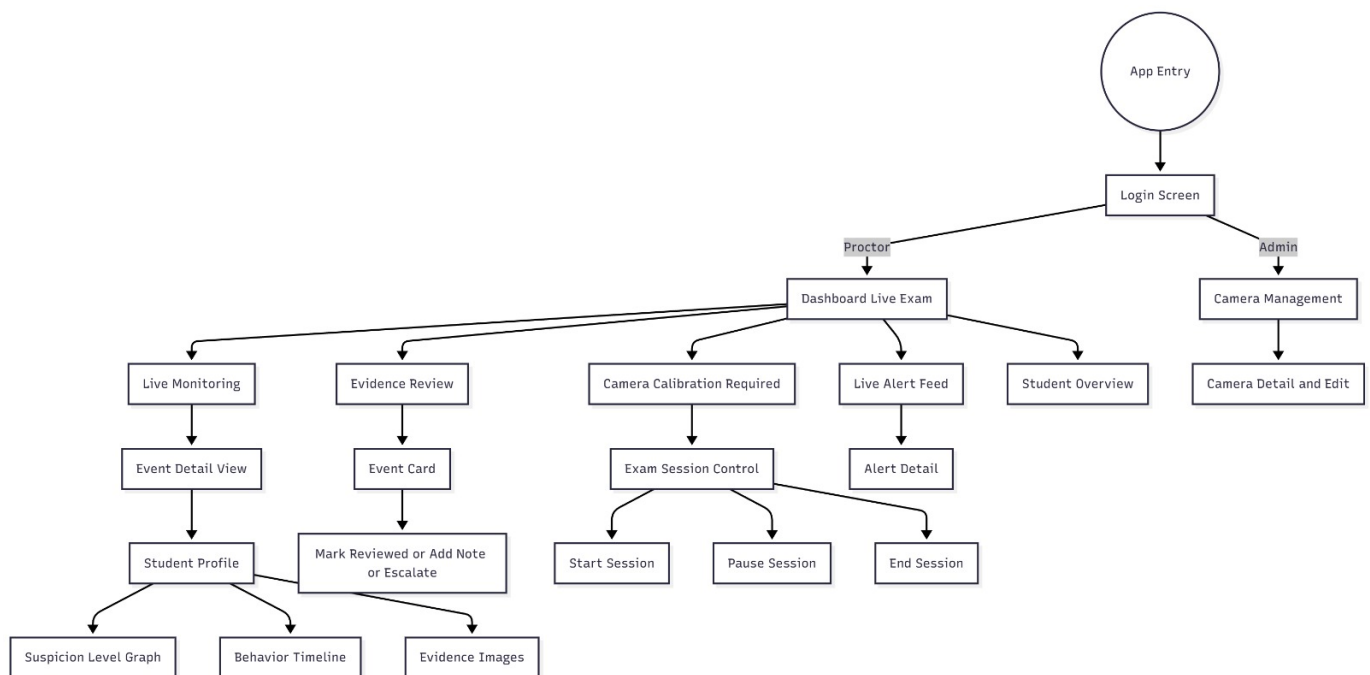
Incident History: Provides access to previously recorded incidents, including video evidence, student identity, and detailed analysis.

Reports Screen: Displays post-exam analysis reports, behavioral summaries, incident statistics, and risk assessment results.

Settings: Allows administrators to configure camera mappings, AI sensitivity levels, data retention policies, and user permissions.

Navigation Flow:

Login → Select Exam Session → Live Monitoring Dashboard → Alerts & Incident Review → Reports & Summary



Navigational Path: The navigational path presents a clear overview of how users interact with the HorusEye system. It ensures that both supervisors and administrators can seamlessly access system features by outlining the flow between key interfaces such as Login, Exam Session Selection, Live Monitoring Dashboard, Alerts, Incident Review, and Report Analysis. This structured navigation helps users efficiently move between real-time monitoring, alert management, and post-exam evaluation processes.

Screen Mock-Ups :

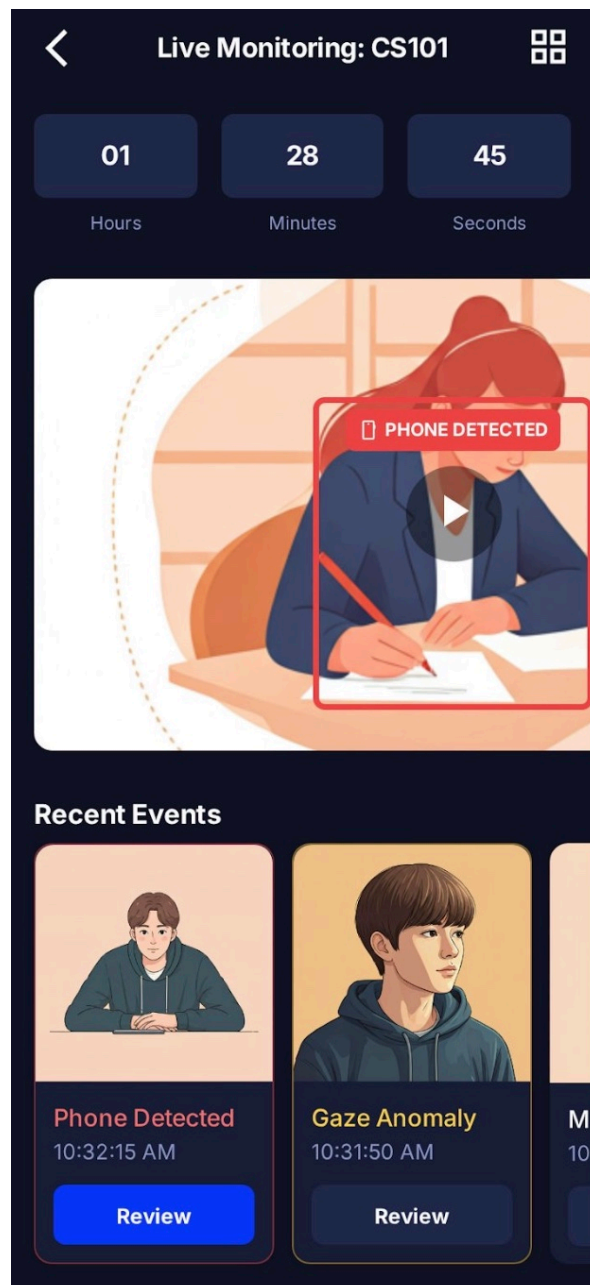
The mockups illustrate the core interface of HorusEye, demonstrating how supervisors interact with the real-time monitoring and incident detection system. The Live Monitoring Dashboard displays active exam sessions with a countdown timer, live camera feed, and automatic behavioral detection such as *Phone Detected* and *Gaze Anomaly*. Detected events are visually highlighted on the video stream and listed under Recent Events, each with timestamp and “Review” option for quick inspection.

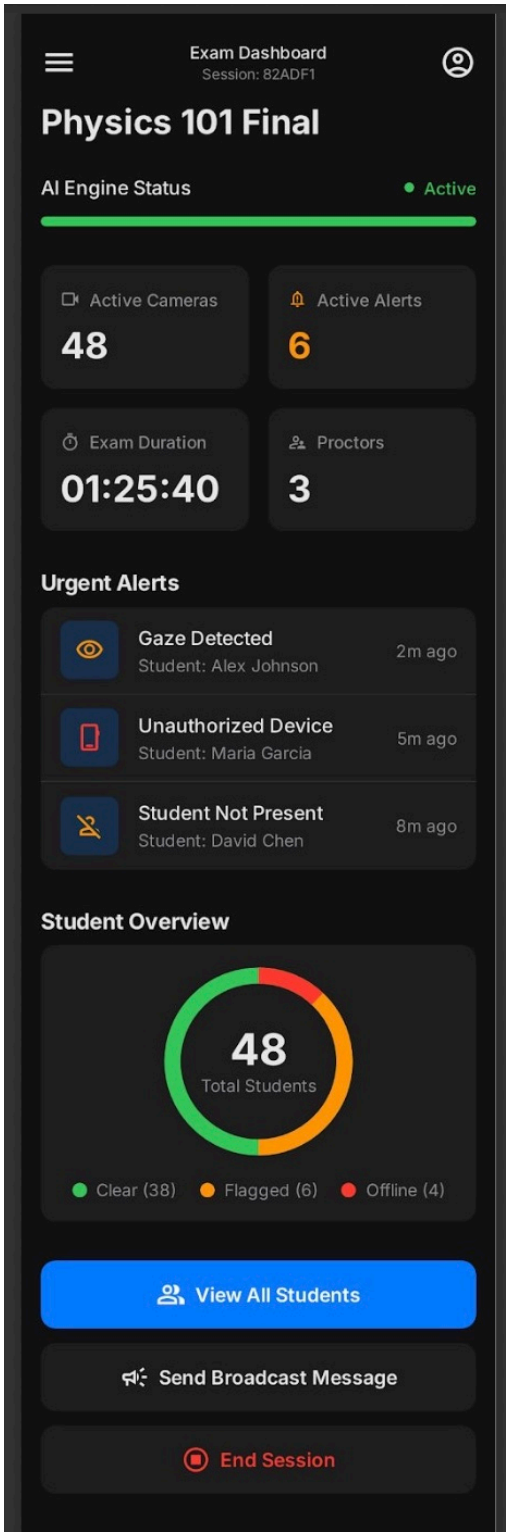
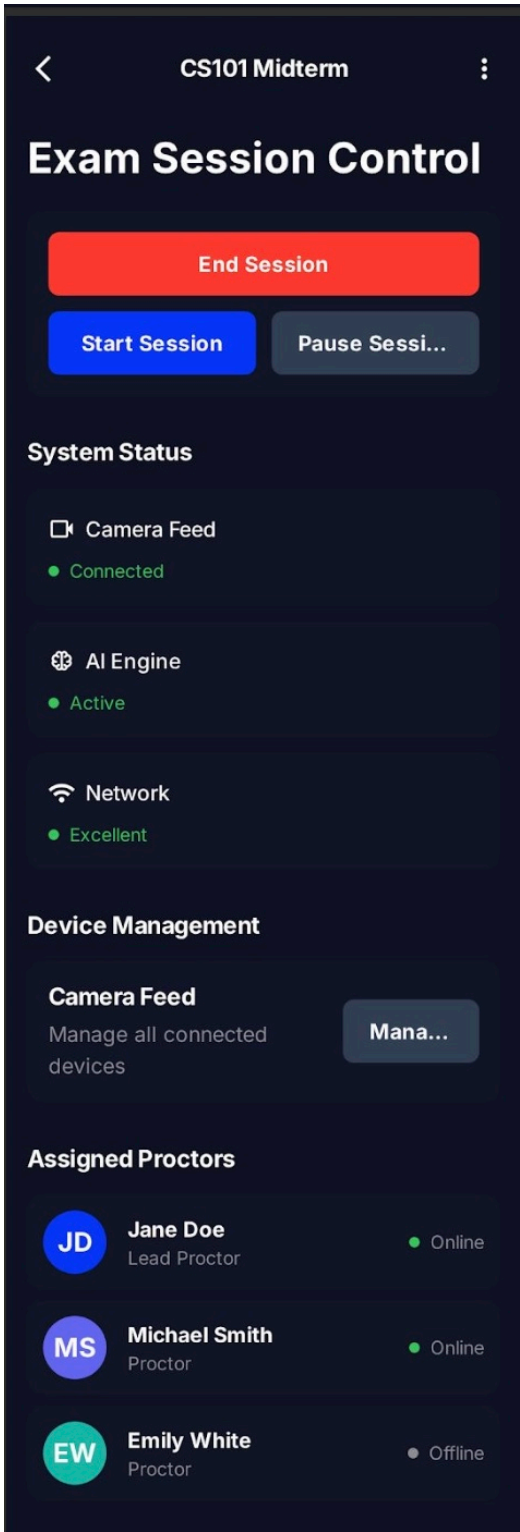
The Exam Session Control Panel allows supervisors to start, pause, and end monitoring sessions while viewing real-time system status, including camera connectivity, AI engine activity, and network quality. Proctors assigned to the session are displayed with their online/offline status, helping coordinate supervision.

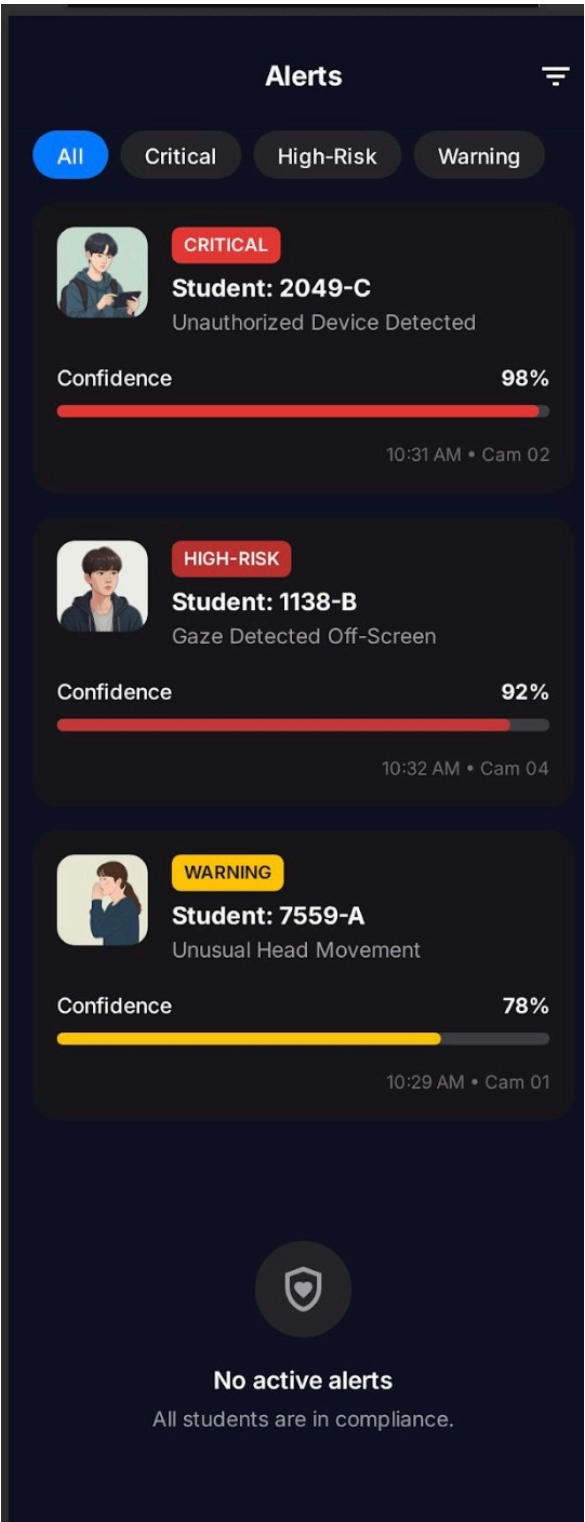
The Exam Dashboard Overview provides a high-level summary of the monitoring environment, including the number of active cameras, total alerts, exam duration, and number of proctors. It also highlights urgent alerts such as *Gaze Detected*, *Unauthorized Device*, and *Student Not Present*. A visual student overview chart presents the distribution of clear, flagged, and offline students.

The Alerts Interface categorizes incidents by severity levels such as *Critical*, *High-Risk*, and *Warning*, supported by AI confidence scores to help supervisors prioritize which events need attention. Each alert contains student identity, event type, time, and camera source.

Finally, the Student Profile View shows a timeline of detected behaviors, suspicion level trends, and video evidence clips for each incident. This allows supervisors to review past actions, analyze patterns, and make informed decisions based on objective, timestamped visual proof.





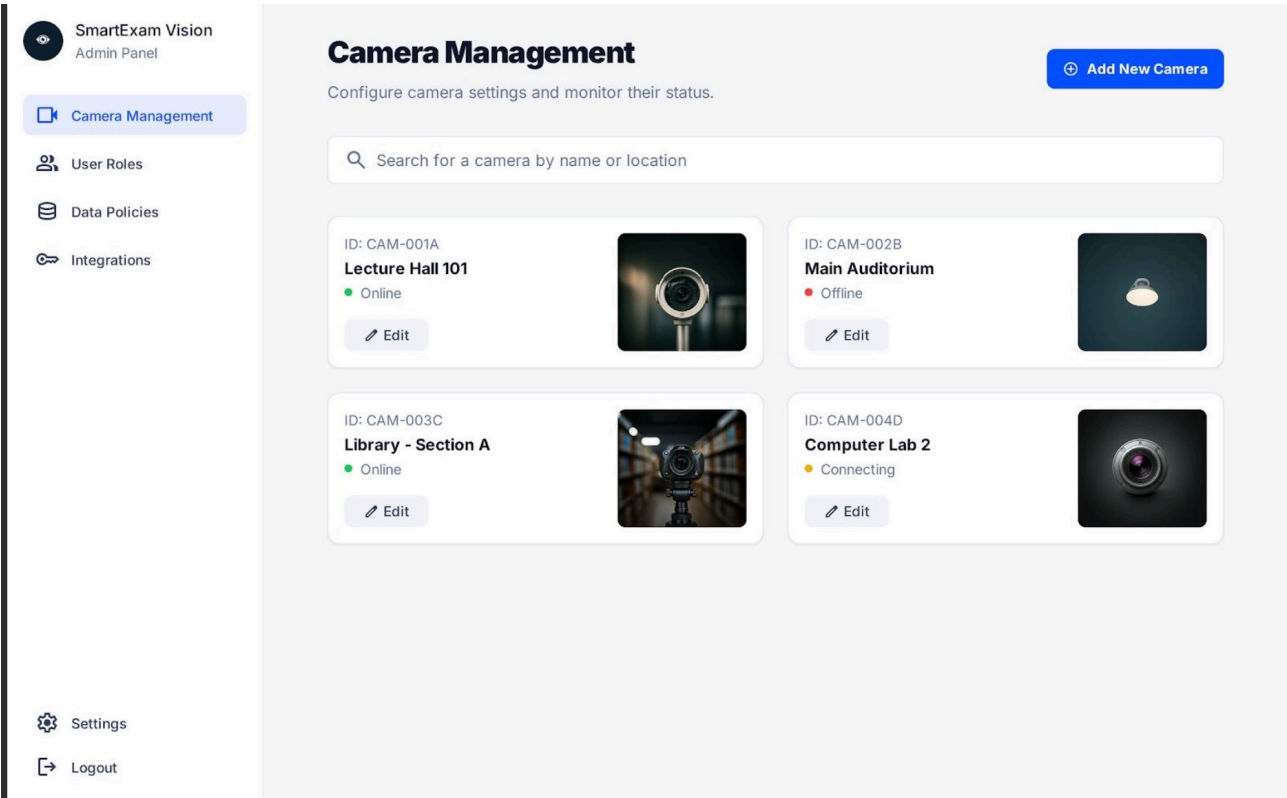


The **Camera Management Screen** allows administrators to view, configure, and monitor the status of all cameras deployed in different exam locations such as lecture halls, auditoriums, libraries, and computer labs. Each camera card displays its ID, location, and connection status (Online, Offline, Connecting), while offering options to edit configurations or add new devices.

Before starting an exam session, the **Camera Calibration Interface** appears, guiding supervisors to ensure proper visibility and camera positioning. The interface provides instructions such as ensuring face visibility, adjusting camera angles, and removing obstructions, helping improve AI accuracy in detecting behaviors.

The **Evidence Review Panel** displays a chronological timeline of detected incidents, where proctors can filter events by student, severity, camera, or time. Each incident includes student details, camera location, severity level (HIGH, MEDIUM, LOW), and visual evidence with actions like *Mark as Reviewed*, *Add Note*, or *Escalate* for further investigation.

The **Live Exam Monitoring Dashboard** provides a real-time overview of exam activity, including the number of students, active alerts, session duration, and average AI confidence score. Camera feeds are shown alongside live detection tags such as *Phone Detected*, *Unusual Head Movement*, or *Eyes Off Screen*. The Classroom Occupancy Map visually highlights seat-based alert zones, helping supervisors quickly identify where suspicious behavior is occurring.



SmartExam Pro

Dashboard

Users

2024 Metrics

Exam Session Control

Manage Exams

2024 Metrics

Live Attendance

Exam Results & Analytics

2024 Metrics

Proctor Tools

Exam Logs

2024 Metrics

Support Center

Account Settings

2024 Metrics

Help Resources

Exam Settings

2024 Metrics

System Reports

2024 Metrics

Start Session

End Session

Pause Session

Resume Session

Exit Exam

Report Issue

Camera Calibration Required

Please confirm your camera is set up correctly before starting the session.



Ensure your face is well-lit and clearly visible.

Position the camera at eye level.

Remove any obstructions from the camera's view.

Calibrate & Start Session

 John Doe
Proctor

Dashboard

Live Proctoring

Evidence Review

Settings

Evidence Review: CS101 Midterm

32 Flagged Events, 15 Reviewed

Filter by Student

Filter by Severity

Filter by Camera

Filter by Time

Timeline of Events

Severity: HIGH

Timestamp: 10:32:15 AM

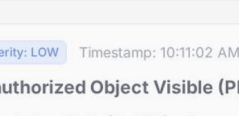
Gaze Detected Off-Screen

Student: Alice Johnson (ID: 1123) • Camera: 04 - Back Right

Mark as Reviewed

Add Note

Escalate



Severity: MEDIUM

Timestamp: 10:25:48 AM

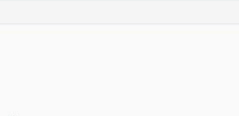
Multiple Faces Detected

Student: Bob Williams (ID: 1124) • Camera: 01 - Front Left

Mark as Reviewed

Add Note

Escalate



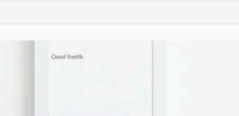
Severity: LOW

Timestamp: 10:11:02 AM

Unauthorized Object Visible (Phone)

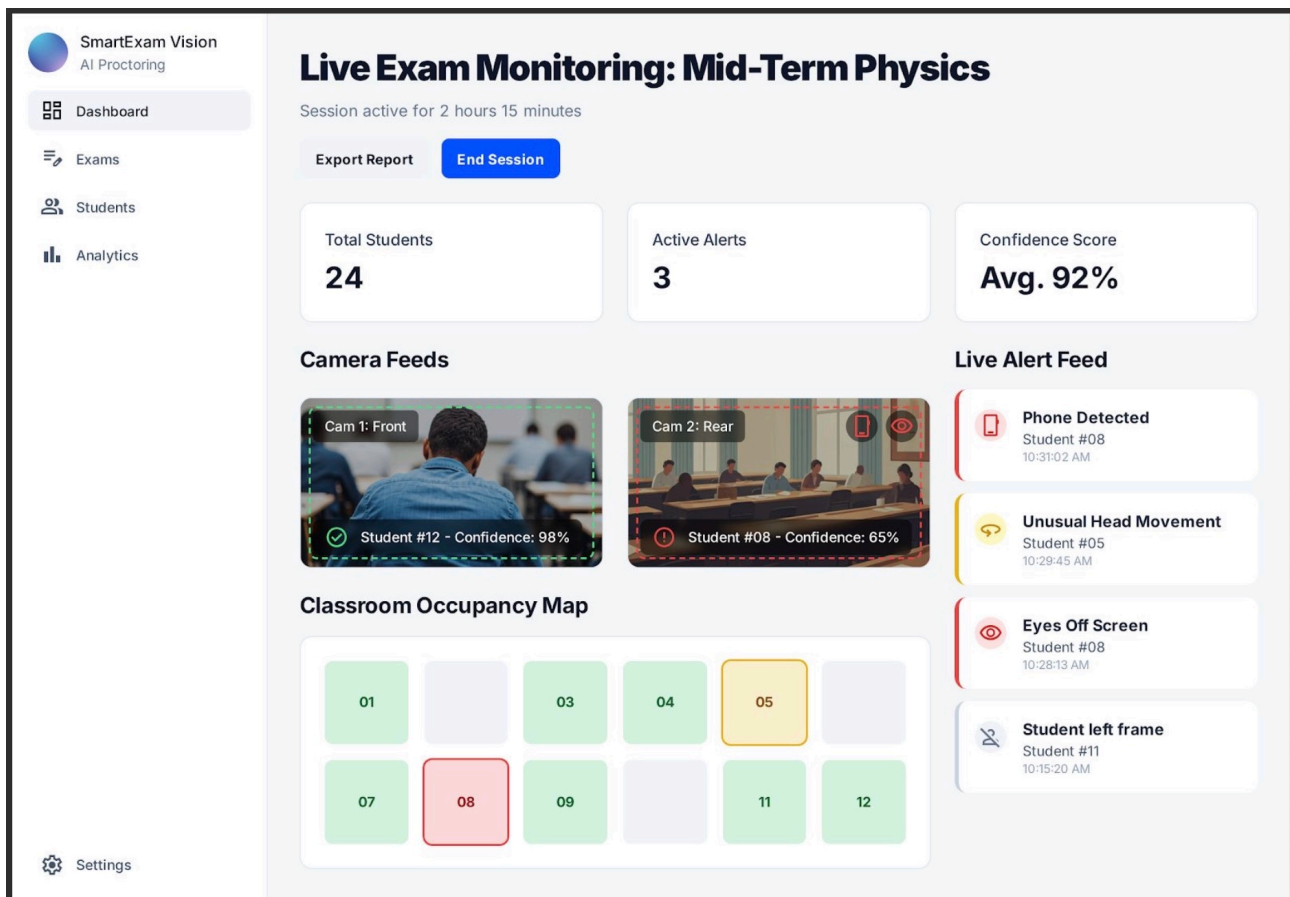
Student: Carol Smith (ID: 1125) • Camera: 12 - Overhead

Reviewed



Start New Exam

[Start New Exam](#)

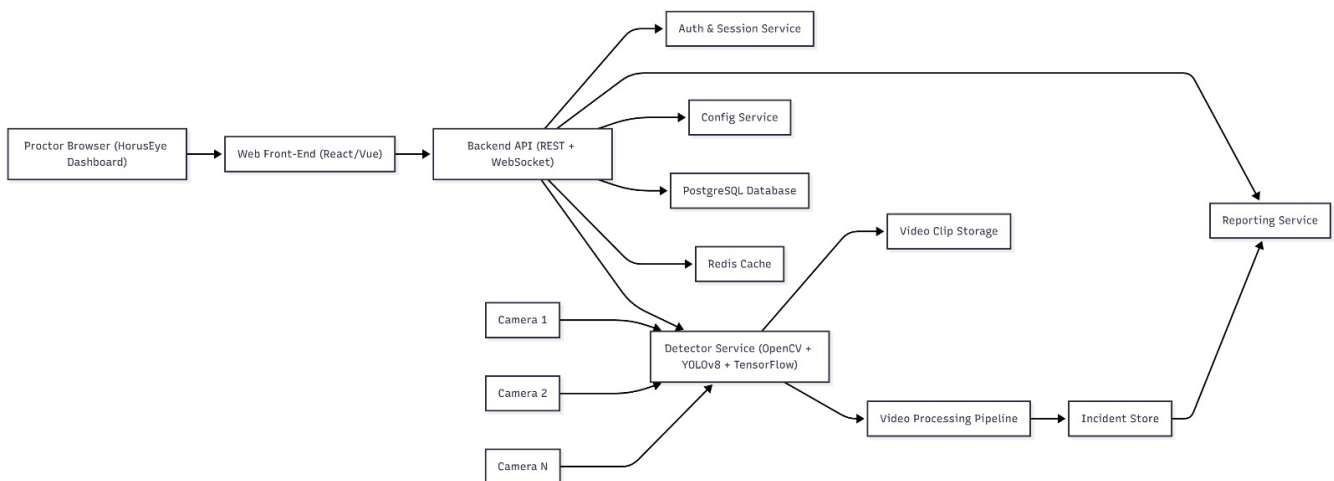


3.5.6 System Architecture Diagram

Layers:

- **Front-End Interface (Web Dashboard):**
Developed using React or Vue.js, it allows supervisors and administrators to access live monitoring screens, manage alerts, review incidents, and generate reports. It communicates with the backend in real time.
- **Backend Services (REST API + WebSocket):**
Handles communication between the dashboard, AI modules, and database. REST API is used for report access, incident history, and system configuration, while WebSocket supports real-time streaming, live alerts, and status updates.
- **AI Processing Layer (Computer Vision & Deep Learning):**
Implements video analysis using OpenCV, YOLOv8, and TensorFlow-based models for face detection, gaze tracking, object recognition, and behavioral analysis. Suspicious behavior events are forwarded to the backend for alert generation.

- **Database Layer (PostgreSQL):**
Stores user information, exam sessions, incident records, timestamps, risk levels, and post-exam reports securely.
- **Cache & Real-Time State Layer (Redis):**
Used for temporary storage of live camera feed metadata, alert queues, session status, and real-time event synchronization to support rapid AI response and low-latency notification delivery.
- **Containerization & Deployment Layer (Docker):**
All system components (AI engine, backend services, frontend, and database) are deployed in Docker containers, enabling scalability, modularity, and easy deployment across local servers or cloud infrastructure.



4. Glossary

Computer Vision:

A field of artificial intelligence that enables HorusEye to interpret and understand visual input from cameras to detect behaviors, objects, and actions during exams.

AI-Based Behavior Detection:

The process where HorusEye uses deep learning to identify suspicious activities such as phone usage, whispering, gaze diversion, or abnormal posture changes.

Exam Session:

A defined time interval during which the system monitors students. It includes exam duration, class location, camera feeds, proctors, and AI detection status.

Suspicious Activity Alert:

An automated notification generated by the system when potential cheating or unusual behavior is detected, sent to supervisors through the dashboard.

Incident Logging:

The process of recording timestamped video evidence, detected behavior type, student ID, and severity level for later review.

Proctor Dashboard:

The web-based interface used by supervisors to monitor live feeds, view alerts, track student behavior, and review evidence in real time.

Confidence Score:

A numerical indicator showing how certain the AI model is about the detected suspicious behavior (e.g., 92% confidence of phone detection).

Camera Calibration:

An initial setup process that ensures cameras have proper positioning, lighting, and face visibility for accurate AI analysis.

Post-Exam Analysis Report:

A detailed summary automatically generated after the exam, including behavior trends, incident frequency, flagged students, and evidence clips.

Real-Time Monitoring:

The live surveillance mode where HorusEye continuously analyzes video feeds and reports suspicious activities instantly.

Student Profile View:

A detailed visualization that displays individual student behavior history, suspicion timeline, and evidence collected during the exam.

Dashboard Notification System:

The interface component that organizes and categorizes alerts based on severity, such as Critical, High-Risk, or Warning.

Microservice Architecture:

A modular backend structure where different services (AI detection, camera management, alert processing, reporting) operate independently to improve scalability and reliability.

Redis Cache:

High-speed temporary data storage used to handle real-time alert traffic, live video status, and session synchronization.